Reproducing and Extending PatchTST: Patch-Based Transformers for Long-Term Time Series Forecasting

Shi Yuan Chu, Ziming Gao, Wenhui Huang, Bing Wen Lim, Yuxiao Wang, Yanting Yang

1 Introduction

Long-term time series forecasting (LTSF) is a critical task in fields ranging from energy grid management to financial market analysis. While Transformer architectures have dominated sequence modeling in natural language processing, their application to LTSF has been hindered by limitations in point-wise tokenization. They treat each individual time step as a token, resulting in $O(L^2)$ attention complexity over the sequence length L. They also have poor inductive bias as a single scalar value provides limited contextual information in isolation.

Early Transformer-based forecasting models attempted to improve efficiency and temporal modeling. Informer introduced ProbSparse attention to reduce the quadratic cost [1], while Autoformer incorporated seasonal-trend decomposition to try and capture long-term temporal patterns [2]. However, neither approach fundamentally departs from point-wise tokenization of time steps. This limitation was empirically highlighted by Zeng et al., who showed that a simple single-layer linear model (DLinear) could match or outperform complex Transformer-based architectures like Autoformer on standard benchmarks [3].

PatchTST addresses this by borrowing an insight from computer vision: just as images are tokenized into patches rather than pixels, time series can be tokenized into contiguous segments of consecutive time steps. Each patch captures a local temporal pattern (a trend, a spike, or a seasonal fragment) rather than a meaningless point, while reducing the number of tokens roughly by a factor of S where S is the stride length. The model further adopts a Channel-Independent (CI) design, where each variable in a multivariate series is processed as a separate univariate sequence through a shared Transformer backbone. This acts as a powerful regularizer and effectively multiplies the available training data by the number of channels. In this project, we reproduce and extend PatchTST to evaluate both its claims and its architectural limitations. Our contributions are:

- **Reproduction from First Principles:** A ground-up PyTorch implementation based on the paper's description, validated on ETTh1 against the paper's reported metrics. We document discrepancies between the paper and the authors' reference code, including the use of RevIN in place of the described instance normalization, and an unconditional padding scheme in the patching formula that we correct.
- **Evaluation on additional datasets:** We evaluate PatchTST, DLinear, and Autoformer on datasets beyond the original benchmarks, including a highly correlated multivariate dataset, to test whether the reported advantages generalize.
- **Cross-Channel Extension:** A cross-channel attention module that relaxes the CI assumption, targeting scenarios where channels share partial temporal patterns and the shared backbone may receive conflicting training signals under strict independence.

2 PatchTST

2.1 Overview

PatchTST is an encoder-only Transformer for multivariate long-term forecasting. Given an input $\mathbf{X} \in \mathbb{R}^{L \times C}$ with lookback length L and C channels, the model predicts $\hat{\mathbf{Y}} \in \mathbb{R}^{H \times C}$ for forecast horizon H. The forward pass consists of seven stages: instance normalization, channel-independent reshaping, patching with embedding, a Transformer encoder, a linear prediction head, reshaping back to multivariate form, and denormalization.

2.2 Instance Normalization

Before entering the model, each sample is independently normalized along the time axis: the per-channel mean and standard deviation are computed, subtracted and divided respectively, and saved for later reversal:

$$\tilde{x}_{t,c} = \frac{x_{t,c} - \mu_c}{\sigma_c}, \qquad \mu_c = \frac{1}{L}\sum_{t=1}^{L} x_{t,c}, \qquad \sigma_c = \sqrt{\frac{1}{L}\sum_{t=1}^{L}(x_{t,c} - \mu_c)^2 + \epsilon} \tag{1}$$

This is distinct from the dataset-level z-score standardization applied during preprocessing, which uses global training set statistics to bring all channels to a common scale. Instance normalization operates per-sample, stripping each individual window's local level and spread so the model focuses on the shape of the signal rather than its absolute position, addressing the non-stationarity inherent in time series data. The statistics are detached from the computation graph—the model cannot learn to manipulate them through backpropagation.

Notably, the official implementation replaces this with RevIN [7], a learned normalization module with trainable affine parameters (γ, β). The paper itself describes only the simpler statistical normalization without learned parameters. We implement the paper's description.

2.3 Channel Independence

The input $\mathbf{X} \in \mathbb{R}^{B \times L \times C}$ is reshaped to $\mathbb{R}^{(B \cdot C) \times L}$ by treating each channel as an independent sample. From this point, the model processes $B \cdot C$ univariate sequences through a shared backbone with no cross-channel interaction. This design rests on the assumption that useful temporal patterns can be learned independently and shared across channels—whether because channels are independent, or because they share sufficiently similar dynamics that a single set of learned temporal patterns serves all channels.

This assumption provides two concrete benefits: regularization (the model cannot overfit to spurious cross-channel correlations) and data efficiency (C times more training sequences from the same data). However, it introduces a failure mode when channels share partial temporal patterns: the shared backbone may receive contradictory gradient signals from channels that look similar in the lookback window but diverge in the prediction horizon. Our extension in Section 5 addresses this limitation.

2.4 Patching

The univariate sequences are segmented into overlapping patches using a sliding window of length P with stride S. The number of patches is:

$$N = \left\lceil \frac{L - P}{S} \right\rceil + 1 \tag{2}$$

When the sequence length does not divide evenly, the final values are replicated to complete the last patch. The original paper defines $N = \lfloor (L-P)/S \rfloor + 2$ and unconditionally pads S

values, which produces an identical result when padding is needed but creates a redundant patch containing fabricated data when the sequence divides cleanly. We adopt the ceiling formulation, which pads only when necessary.

Patching reduces the attention complexity from $O(L^2)$ to $O(N^2)$. With typical settings (L=336, P=16, S=8), this reduces the token count from 336 to 41 (an ≈8× reduction), and the quadratic attention cost by approximately 67×.

2.5 Patch Embedding

Each patch of P raw values is projected to a D-dimensional representation via a shared linear layer. A learnable positional encoding is added to each patch position, providing the model with ordering information that the permutation-invariant attention mechanism would otherwise lack:

$$\mathbf{z}_i = \text{Dropout}(\mathbf{W}_{\text{proj}}\,\mathbf{p}_i + \mathbf{e}_i) \tag{3}$$

where $\mathbf{e}_i$ is the i-th row of a learnable positional embedding matrix $\mathbf{E}_{\text{pos}} \in \mathbb{R}^{N\times D}$.

2.6 Transformer Encoder

The encoder consists of M identical layers, each containing two sub-blocks:

1. **Multi-head self-attention**: The input is projected into queries, keys, and values via learned linear transformations, then split across H attention heads each operating in D/H dimensions. Each head computes scaled dot-product attention over the same set of N patch tokens in a different projected subspace, then the heads are concatenated and projected back to D dimensions.

2. **Position-wise feed-forward network**: A two-layer MLP ($D \to d_{ff} \to D$) with GELU activation expands and compresses each token's representation independently.

Both sub-blocks use residual connections followed by BatchNorm—a departure from the LayerNorm used in standard Transformers. The normalization is applied post-residual (post-norm), meaning the residual is added before normalization:

$$\mathbf{z}' = \text{BatchNorm}(\mathbf{z} + \text{Dropout}(\text{MHA}(\mathbf{z}))) \tag{4}$$

$$\mathbf{z}'' = \text{BatchNorm}(\mathbf{z}' + \text{Dropout}(\text{FFN}(\mathbf{z}'))) \tag{5}$$

In a channel-independent setting, the effective batch size is $B \cdot C$ (e.g., $128 \times 7 = 896$), providing the statistical stability that BatchNorm requires.

2.7 Prediction Head and Denormalization

The encoder output $\mathbb{R}^{(B\cdot C)\times N\times D}$ is flattened to $\mathbb{R}^{(B\cdot C)\times(N\cdot D)}$ and projected to the prediction horizon H via a single linear layer with no hidden layers or activation function: $\hat{\mathbf{y}} = \mathbf{W}_{\text{head}}[\mathbf{z}_1; \ldots; \mathbf{z}_N] + \mathbf{b}$. The output is reshaped back to $\mathbb{R}^{B\times H\times C}$ and denormalized using the per-sample statistics saved during instance normalization, restoring the predictions to the original data scale.

3 Reproduction

We implemented PatchTST in PyTorch based on the paper's description, cross-referencing the authors' source code [7] where the paper was ambiguous. The core architecture—patching, multi-head attention, and encoder—was built from scratch; standard components such as the feed-forward dropout placement were adopted from the reference implementation. DLinear and

Autoformer were adopted from Time-Series-Library [8]. To validate correctness, we trained all three models on ETTh1 (7 variables, hourly, 17,420 time steps, standard 12/4/4 month split) using the authors' ETTh1 configuration: L=336, T=96, P=16, S=8, D=16, H=4, d_{ff}=128, M=3 layers, dropout 0.3, Adam (lr=10^{-4}, constant for 3 epochs then 0.9× decay per epoch following the reference code), batch size 128. We deviated from the paper's settings in two respects: seed 42 (vs. 2021) and early stopping patience 10 (vs. 100).

During reproduction, we identified four discrepancies between the paper and its reference code: (1) the patching formula unconditionally pads S values, producing a redundant patch when the sequence divides evenly—we corrected this as described in Section 2.4; (2) the official implementation replaces instance normalization with RevIN, a learned normalization module—we followed the paper; (3) the paper text (Appendix A.1.4) reports dropout 0.2 while the authors' ETTh1 training script uses 0.3—we used 0.3; (4) the reference code applies dropout twice on the attention output—once after the output projection inside the attention module, and again on the residual path in the encoder layer—resulting in an effective drop rate of $1-(1-p)^2 \approx 0.51$ for p=0.3. The paper describes a single dropout per sub-block. We removed the redundant projection dropout to match the paper's description.

Table 1: ETTh1 reproduction validation (pred_len = 96). Lower is better.

Model	MSE (Paper)	MSE (Ours)	MAE (Paper)	MAE (Ours)	Train Time
PatchTST	0.375	0.381	0.399	0.403	196s
DLinear	0.375	0.374	0.399	0.397	9s
Autoformer	0.435	0.528	0.446	0.491	1205s

PatchTST and DLinear are within 2% of the paper's reported MSE (Table 3, PatchTST/42 and DLinear for ETTh1 96 metric, in [4]), confirming faithful implementations. The Autoformer gap is expected: the paper swept six lookback lengths ($L \in \{24, 48, 96, 192, 336, 720\}$) and reported the best result; we only evaluated at L=336.

4 Evaluation on Additional Datasets

To evaluate the generalization capability of PatchTST beyond the ETTh1 benchmark, we conducted experiments on additional datasets with diverse statistical properties. We selected benchmark datasets with seasonal patterns analogous to, but not included in, the original paper. Furthermore, We extended to focus on dataset with strong cross-channel dependencies and weak temporal regularity, which present a challenging setting for models that rely on local temporal patterns and channel-independent representations.

Specifically, we consider three datasets:

- **Traffic Dataset:** With strong daily patterns and spatial dependencies.
- **Air Quality Dataset:** Moderate seasonality with exogenous coupling.
- **Exchange Rate Dataset:** With high volatility and weak seasonality.

4.1 Experimental Setup

4.1.1 Sequence Construction

A sliding window approach is used to construct samples:

- Input sequence length: $L = 336$

- Prediction horizon: $H = 96$
- Label length: 48

Each sample consists of:

- Encoder input: past 336-timesteps
- Decoder target: next 96-timesteps

4.1.2 Train/Validation/Test Split

Chronological split:

- 70% training
- 15% validation
- 15% test

To preserve temporal continuity, validation and test splits are shifted backward by the lookback length. This ensures that all sliding windows remain valid without losing historical context.

4.1.3 Normalization

We apply feature-wise z-score normalization using training data only:

$$x_{\text{normalized}} = \frac{x - \mu_{\text{train}}}{\sigma_{\text{train}}} \tag{6}$$

Mean and standard deviation are computed exclusively from the training set. Features with zero variance are stabilized by setting σ_{train} to 1. This prevents data leakage and ensures fair evaluation.

4.1.4 Time Feature Encoding

Temporal features are extracted from the datetime index using:

- hour of day
- day of week
- day of month
- day of year

All features are normalized to $[-0.5, 0.5]$.

4.1.5 Model Configuration

All models are trained under consistent settings:

- Input sequence length: 336
- Prediction horizon: 96
- Batch size: 128
- Optimizer: Adam

- Learning rate: 1×10^{-4}
- Learning rate schedule: constant for the first three epochs, followed by exponential decay with factor 0.9 per epoch
- Maximum epochs: 100
- Early stopping patience: 10

PatchTST Configuration:

- Patch length: 16
- Stride: 8
- Embedding dimension: 128
- Attention heads: 16
- Encoder layers: 3
- Feed-forward dimension: 256
- Dropout: 0.2

The best model is selected based on validation MSE.

4.1.6 Baselines

We compare PatchTST against:

- DLinear — a linear model with strong empirical performance in time-series forecasting
- Autoformer — a Transformer-based architecture with seasonal-trend decomposition

This provides a comparison across both simple and complex model classes.

4.1.7 Evaluation Metrics

We report:

- **Mean Squared Error (MSE):** $\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2$
- **Mean Absolute Error (MAE):** $\frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$

All results are computed on the test set using the best validation checkpoint.

4.2 Additional Benchmark Dataset 1: Traffic Dataset

The traffic dataset contains hourly volume measurements on westbound I-94 in Minneapolis-St. Paul (2012-2018), along with weather and holiday features [5].

After preprocessing, the dataset contains:

- Length: 48,204-timesteps
- Frequency: Hourly
- Input features: 20 variables (including feature engineered and one-hot encoded)

4.2.1 Key Characteristics

- Rich contextual features: Includes weather conditions (temperature, precipitation, cloud cover) and holiday indicators, making it suitable for predictive modeling.
- Cyclical and seasonal patterns: Captures recurring annual events (holidays, seasonal weather shifts) that systematically influence traffic volume across time.

This dataset serves as a performance benchmark to verify PatchTST's results presented in the paper.

4.2.2 Performance Comparison

The results on the traffic dataset demonstrate that PatchTST consistently outperforms both baseline models, though the margins are narrow.

- PatchTST obtains the lowest errors on both metrics, with an MSE of 0.538 and MAE of 0.461, indicating the most accurate overall predictions.
- Compared to DLinear, PatchTST reduces MSE more by 7.4% and MAE by 10.8%.
- Autoformer performs moderately among the three models. While its decomposition mechanism helps capture trend and seasonality, it remains behind PatchTST by 4.8% in MSE and 9.1% in MAE, indicating less precise overall predictions.

Table 2: Performance comparison on the traffic dataset. Lower is better.

Model	MSE	MAE
PatchTST	0.538	0.461
DLinear	0.581	0.517
Autoformer	0.565	0.507

Traffic volume's strong daily/weekly periodicity and autoregressive structure (e.g. current volume depends heavily on the previous hour) allows simpler models like DLinear to perform competitively. However, PatchTST's patch-based representation still provides an advantage by capturing fine-grained local temporal dependencies that are not fully exploited by the other models.

4.3 Additional Benchmark Dataset 2: Air Quality Dataset

The air quality dataset consists of hourly PM2.5 concentration measurements collected at the US Embassy in Beijing, covering the period from January 1, 2010 to December 31, 2014[6]. To provide contextual environmental information, the dataset also includes meteorological data obtained from Beijing Capital International Airport.

After preprocessing, the dataset contains:

- Length: 43,8244-timesteps
- Frequency: Hourly
- Input features: 20 variables (including feature engineered and one-hot encoded)

4.3.1 Key Characteristics

- Seasonal and daily periodicity: weak but observable patterns, such as diurnal cycles and seasonal trends.
- Volatility and extreme events: contains highly fluctuating pm2.5 values, including extreme spikes and sudden pollution events or sensor anomalies.
- Non-stationarity: Statistical properties shift over time due to changing weather patterns, seasonal transitions, and potential policy interventions.

With particular emphasis on its periodic yet non-stationary dual nature, the dataset constitutes a moderately challenging benchmark to evaluate whether PatchTST can appropriately perform to capture seasonal cycles without encroaching upon incompatible temporal regimes.

4.3.2 Performance Comparison

The results on the air quality dataset show a more significant performance gap compared to the traffic dataset by PatchTST.

- PatchTST make substantially more accurate predictions as evidenced by the lowest error across both metrics, with an MSE of 0.212 and MAE of 0.190.
- Compared to DLinear, PatchTST achieves a significantly larger proportional reduction, lowering MSE by 24.0% and MAE by 34.3%.
- Autoformer performs similarly to DLinear but remains substantially behind PatchTST, which outperforms it by 24.3% in MSE and 33.8% in MAE.

Table 3: Performance comparison on the traffic dataset. Lower is better.

Model	MSE	MAE
PatchTST	0.212	0.190
DLinear	0.279	0.289
Autoformer	0.280	0.287

With the air quality data lacking such strong periodic regularity and exhibiting more complex, non-stationary dynamics, it is harder for simpler or decomposition-based models to capture underlying patterns effectively. This setting confirms PatchTST's competencies in local pattern extraction.

4.4 Highly Correlated Dataset 1: Foreign Currency Exchange Rates

The exchange rate dataset consists of daily foreign exchange rates over 5,978-timesteps. Each variable represents the exchange rate of a currency with a total of 42 currency exchange rates.

After preprocessing, the dataset contains:

- Length: 5,978-timesteps
- Frequency: Daily
- Input features: 42 currency exchange rates

4.4.1 Key Characteristics

- High cross-variable correlation: Exchange rates are jointly influenced by global macroeconomic factors, resulting in strong co-movement across currencies.
- Weak periodicity: Seasonal patterns are unstable.
- High volatility and noise: Exhibit stochastic fluctuations driven by market dynamics.
- Non-stationarity: Structural changes over time (e.g., financial crises) alter statistical properties.

These characteristics make the dataset particularly suitable for evaluating the limitations of channel-independent modelling.

4.4.2 Performance Comparison

The results demonstrate a clear performance advantage of PatchTST over both baseline models.

- PatchTST achieves the lowest error across all metrics, significantly outperforming DLinear and Autoformer.
- Compared to DLinear, PatchTST reduces MSE by approximately 48%, indicating the benefit of modeling non-linear temporal dependencies.
- Autoformer performs substantially worse, suggesting that its decomposition-based architecture is not well-suited to financial time series with weak seasonality.

These results demonstrate that PatchTST is highly effective even in challenging financial time series.

Table 4: Performance comparison on the exchange rate dataset. Lower is better.

Model	MSE	MAE
PatchTST	0.081	0.174
DLinear	0.156	0.260
Autoformer	0.350	0.406

5 Extension: Cross-Channel Attention

TODO: This section will be completed by [teammate names]. Propose and evaluate an adaptive cross-channel attention module that relaxes the strict channel-independence assumption. Include motivation, architecture design, ablation studies, and results.

6 Conclusion

TODO: To be written after all experimental sections are complete.

References

[1] H. Zhou et al., "Informer: Beyond Efficient Transformer for Long Sequence Time-Series Forecasting," *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 35, no. 12, pp. 11106–11115, May 2021, doi:10.1609/aaai.v35i12.17325.

[2] H. Wu, J. Xu, J. Wang, and M. Long, "Autoformer: Decomposition Transformers with Auto-Correlation for Long-Term Series Forecasting," *arXiv preprint arXiv:2106.13008*, Dec. 2021, doi:10.48550/arxiv.2106.13008.

[3] A. Zeng, M. Chen, L. Zhang, and Q. Xu, "Are Transformers Effective for Time Series Forecasting?" *Proceedings of the AAAI Conference on Artificial Intelligence*, vol. 37, no. 9, pp. 11121–11128, Jun. 2023, doi:10.1609/aaai.v37i9.26317.

[4] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, "A Time Series is Worth 64 Words: Long-term Forecasting with Transformers," *International Conference on Learning Representations (ICLR)*, 2023, doi:10.48550/arXiv.2211.14730.

[5] J. Hogue, "Metro Interstate Traffic Volume," UCI Machine Learning Repository, 2019, doi:10.24432/C5X60B.

[6] S. Chen, "Beijing PM2.5 [Dataset]," UCI Machine Learning Repository, 2015, doi:10.24432/C5JS49.

[7] Y. Nie, N. H. Nguyen, P. Sinthong, and J. Kalagnanam, PatchTST: Official implementation, [Online]. Available: `https://github.com/yuqinie98/PatchTST`. [Accessed: Apr. 18, 2026]

[8] Time-Series-Library, [Online]. Available: `https://github.com/thuml/Time-Series-Library`. [Accessed: Apr. 18, 2026]